Assignment 1 - CUDA and Bender Setup

> DUE Monday, October 12 @ 11:59:59PM Pacific Time

Overview

The objective of this assignment is to get set up with CUDA on the Bender ENGR server and write your first CUDA program: element-wise matrix addition, C = A + B.

This assignment uses GPU resources on `bender.engr.ucr.edu`. You can access this through ssh by running the following command in a terminal `ssh <engr-username>@bender.engr.ucr.edu`.

If you are unfamiliar with the Linux command line, there's a great tutorial here: https://www.codecademy.com/learn/learn-the-command-line

1. For this lab, we will be using Classroom 50 (replacement for GitHub Classroom). Please join the classroom assignment by clicking the following link: https://classroom50.org/UCR-CSEE217/cs-ee217-fall-2026/assignments/matrix-addition/accept Once you accept the assignment, a private GitHub repository will automatically be created with the starter code. Simply `git clone` it to copy the starter code to Bender.

Matrix Addition

Once you have access to Bender and have confirmed it can compile and run CUDA code, you will implement matrix addition.

1. Clone the git repository. It contains:
   - `kernel.cu`, `main.cu`: where you write your code
   - `support.cu`, `support.h`, `Makefile`: provided; do not modify
   - `REPORT.md`, `plots/`: your written report (see below)
2. Add the CUDA tools (`nvcc`, `compute-sanitizer`) to your PATH. You only need to do this once:

```
echo 'export PATH=/usr/local/cuda/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
nvcc --version   # should print the CUDA compiler version
```

   If you already set this up while following the Bender tutorial, skip this step.

3. Complete the application by filling in the `INSERT CODE HERE` sections of `main.cu` (allocating device memory, copying data to and from the GPU, freeing memory) and `kernel.cu` (the kernel and its launch). Size your thread blocks with the provided `BLOCK_X` and `BLOCK_Y` (16x16 by default; see Changing the matrix size and block size). You may consult the class slides on vector add. Your code must work for any matrix shape, not just square ones.
4. Build and test on Bender:

```
make
./mat-add             # 1000 x 1000 matrices
./mat-add 1024        # 1024 x 1024
./mat-add 4097 1023   # 4097 rows x 1023 columns
```

   The program checks every element of your result against the CPU and prints `TEST PASSED`, or lists the first wrong elements. To check for out-of-bounds memory accesses, which can go unnoticed otherwise, run:

```
compute-sanitizer ./mat-add 1023 517
```

Changing the matrix size and block size

Matrix size is set on the command line when you run the program; no rebuild is needed:

Command	Matrices
`./mat-add`	1000 x 1000 (default)
`./mat-add 2048`	2048 x 2048
`./mat-add 2048 512`	2048 rows x 512 columns

Block size is set when you build. `basicMatAdd()` in `kernel.cu` provides `BLOCK_X` and `BLOCK_Y`, the number of threads per block in x and y (16 x 16 by default). Use them for your block and grid dimensions, not hard-coded numbers; otherwise changing the block size has no effect. `make` rebuilds automatically whenever a block-size setting changes.

- **Square blocks:** set both dimensions with `TILE`:

```
make TILE=8     # 8 x 8 blocks
./mat-add 4096
make TILE=32    # 32 x 32 blocks
./mat-add 4096
```

- **Non-square blocks:** set each dimension with `TILE_X` and `TILE_Y`:

```
make TILE_X=32 TILE_Y=8    # 32 x 8 blocks
./mat-add 4096
```

- **Back to the default:** plain `make` builds with 16 x 16 blocks again.

Automated tests

Every time you push to `main`, GitHub automatically builds your code and tests several matrix sizes (square and non-square), plus a memory-access check. Open the **Actions** tab of your repository to see the results. If a test fails, click on it: the summary explains the cause (wrong results, a CUDA error, an invalid memory access, or a timeout), and the log shows your program's full output.

Report

Answer the questions in `REPORT.md` in your repository. Expand "How to use this file" at the top of it for how to add plots, tables and math. Some questions ask you to measure and plot your program's performance. You collect the timings and make the plots yourself, with any tool you like. Save the plots as PNG files in `plots/` and embed them in `REPORT.md`.

Some questions ask you to try other matrix sizes and block sizes and shapes; see Changing the matrix size and block size. **Submit your code with the default 16x16 blocks**: don't change the `TILE_SIZE`, `TILE_X` or `TILE_Y` defaults in `kernel.cu`. The automated tests build with a plain `make`.

Submission

1. Fill in your name and answers in `REPORT.md`, and add your plots (if any) to `plots/`.

2. Commit and push your code, `REPORT.md` and `plots/` to your GitHub repository. Open `REPORT.md` on GitHub to check that your plots display; that's the version we grade.
3. Check that the automated tests pass on your final commit.

© 2019 - 2026 Daniel Wong